FPGA 디지털 시계 및 스톱워치 설계 및 UART 통신 구현

김민수

목차

1 프로젝트 개요

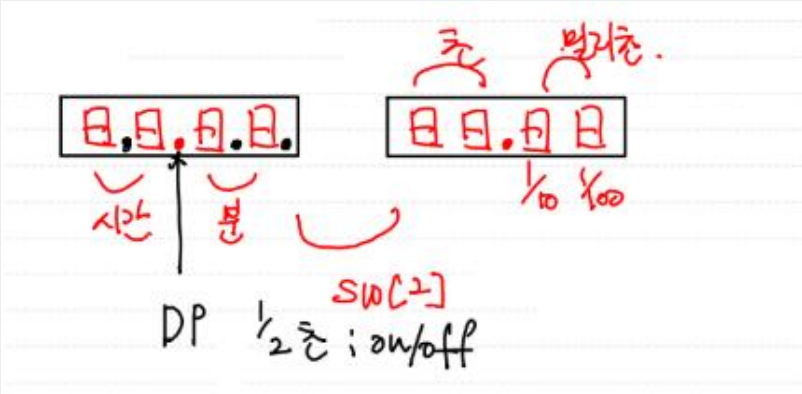
2 시스템 구조

3 상세 설계

4 기능 검증(영상 시연)

5 결론 및 배운 점

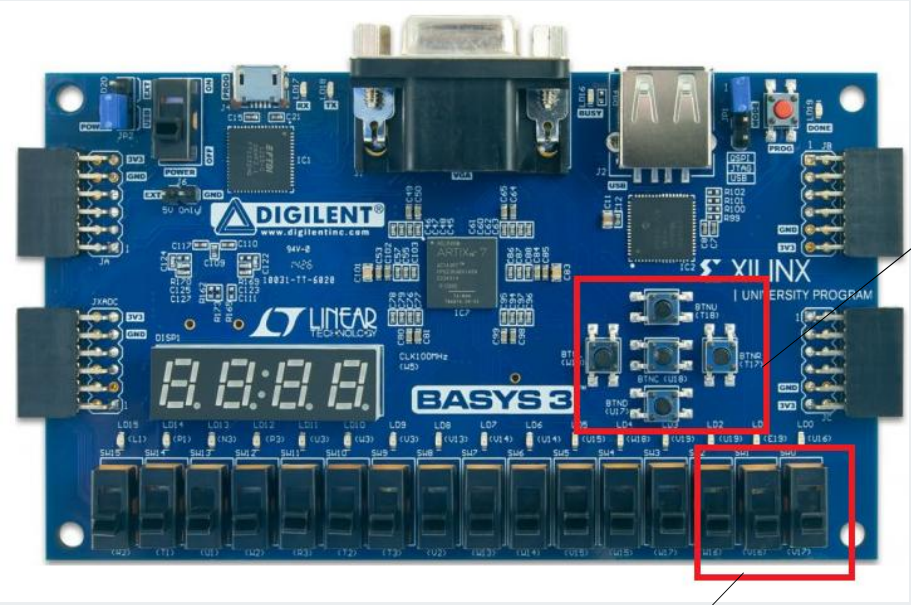
프로젝트 개요



Verilog HDL을 이용한
다기능 Watch & Stopwatch 설계

- 시계 기능 : 초/분/시간 조절 기능
- 스톱워치 기능 : clear(초기화), run_stop(정지/재생), Up down 기능
- DP : 작동 중임을 알 수 있게 0.5초마다 깜빡이는 기능

프로젝트 개요

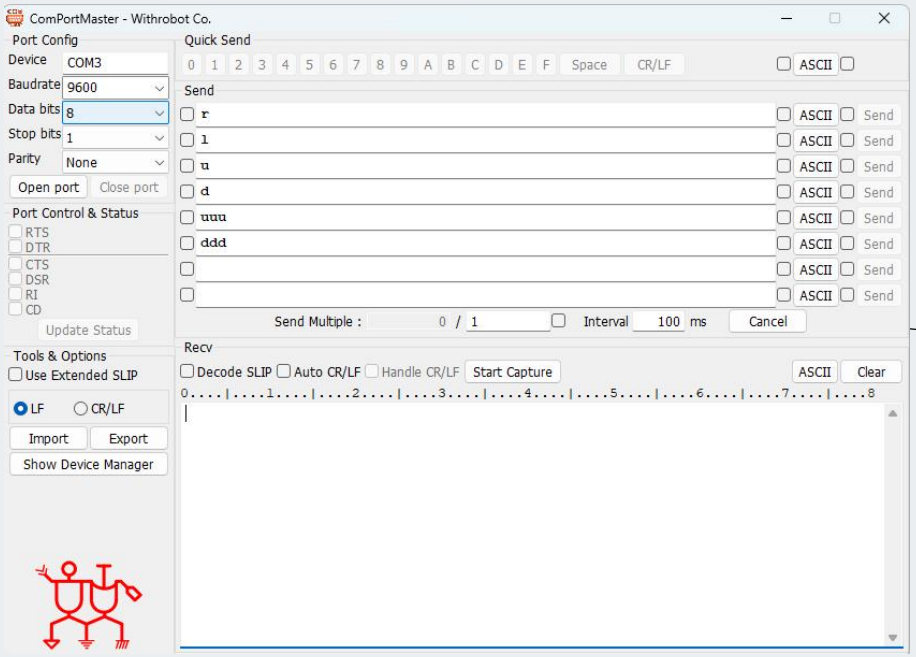


스톱워치 : BTNR로 정지/작동
BTNL로 초기화(이후 바로 동작)

시계 : BTNR, BTNL으로 커서 옮기기(제어할 시간 값)
BTNU, BTNC로 증가 감소 제어

SW[0] == 0 일 경우 증가 / 1이면 감소
SW[1] == 0 일 경우 시계 / 1이면 스톱워치
SW[2] == 0 일 경우 HOUR_MIN / 1이면 SEC_MSEC

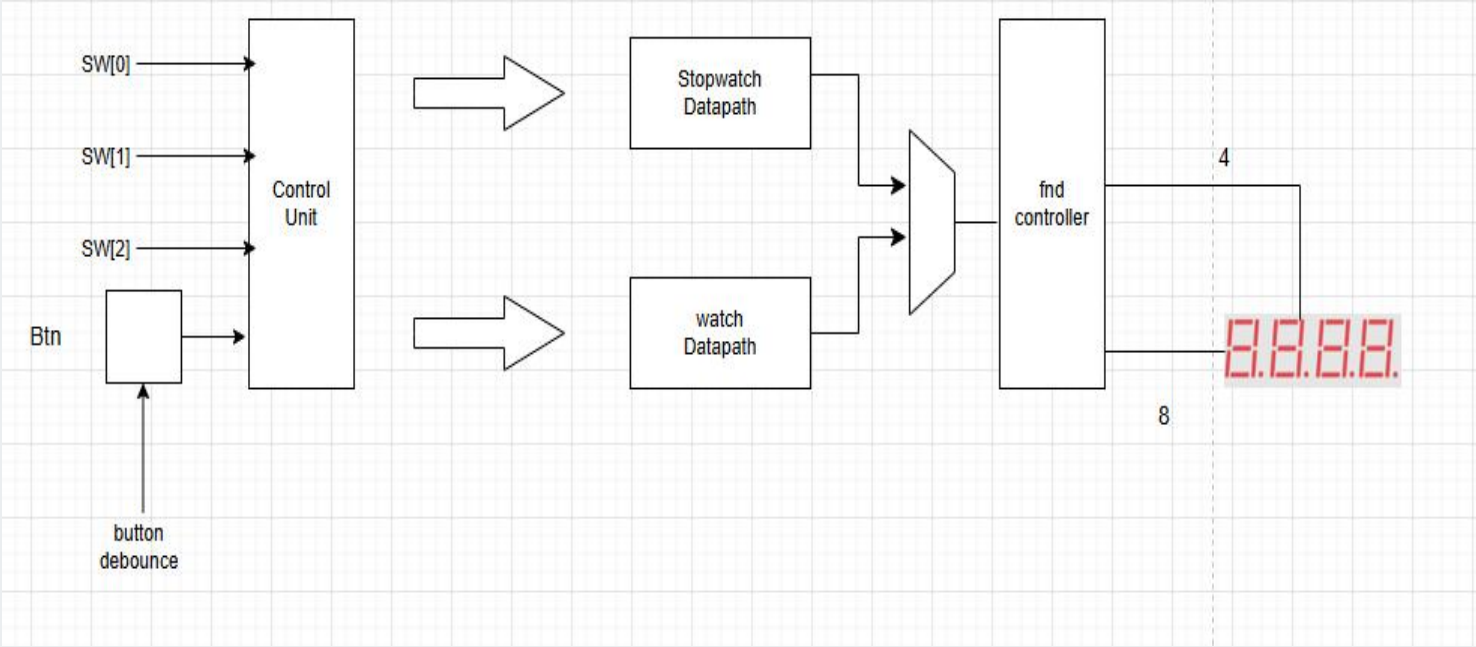
프로젝트 개요



UART 통신을 이용해 PC로 FPGA 제어

시스템 구조

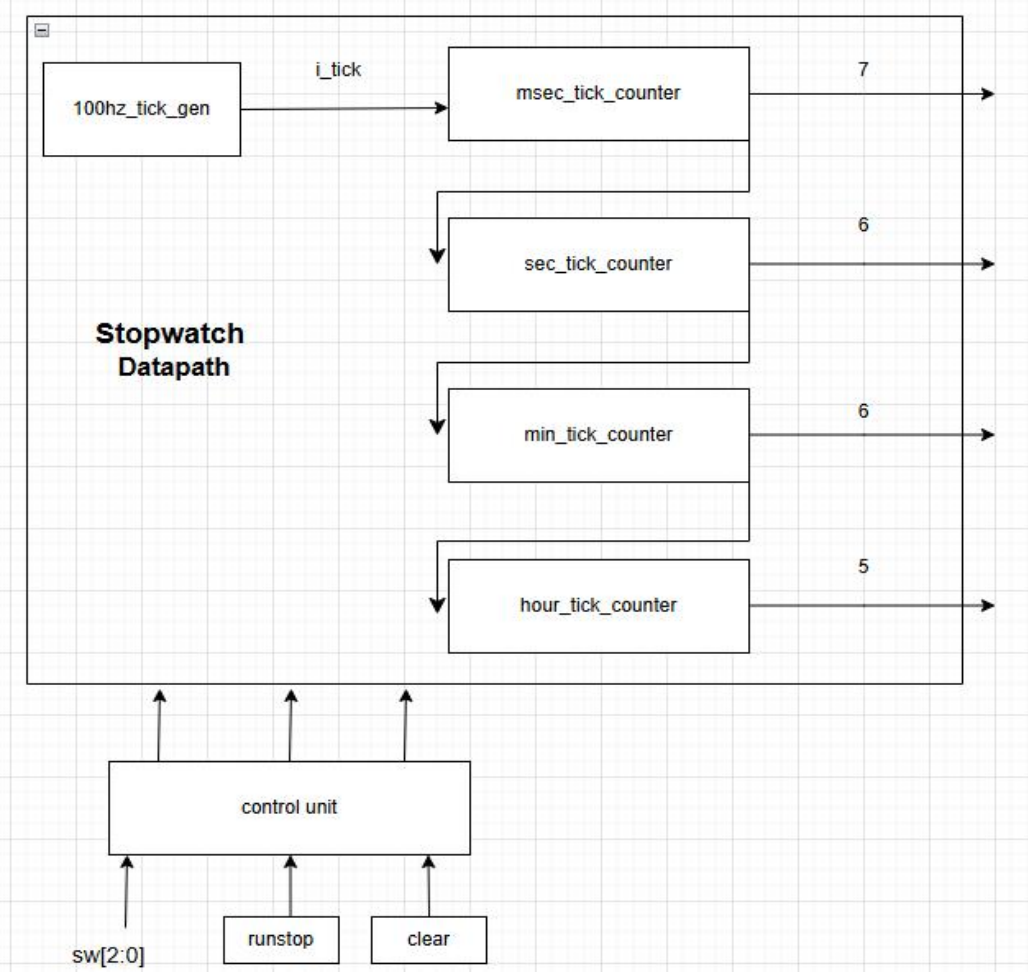
화면에 표시되는 구조



버튼과 스위치를 받는 입력부와 함께 독립적으로 동작하는 시계와 스톱워치 데이터 패스

MUX를 활용해 화면에 보여줄 데이터를 선택 가능

시스템 구조



Msec를 세기 위해 100hz tick 신호 생성

Clock divider를 이용해
1억(100Mhz) 번 진동하는 시스템 클럭을
100만 번 세어 신호를 만들어
0.01초의 시간 간격을 지니게 함

밀리초 100번 > sec_counter 동작
> 60번 돌면 > min_counter 동작 > hour 동작
의 구조로 설계

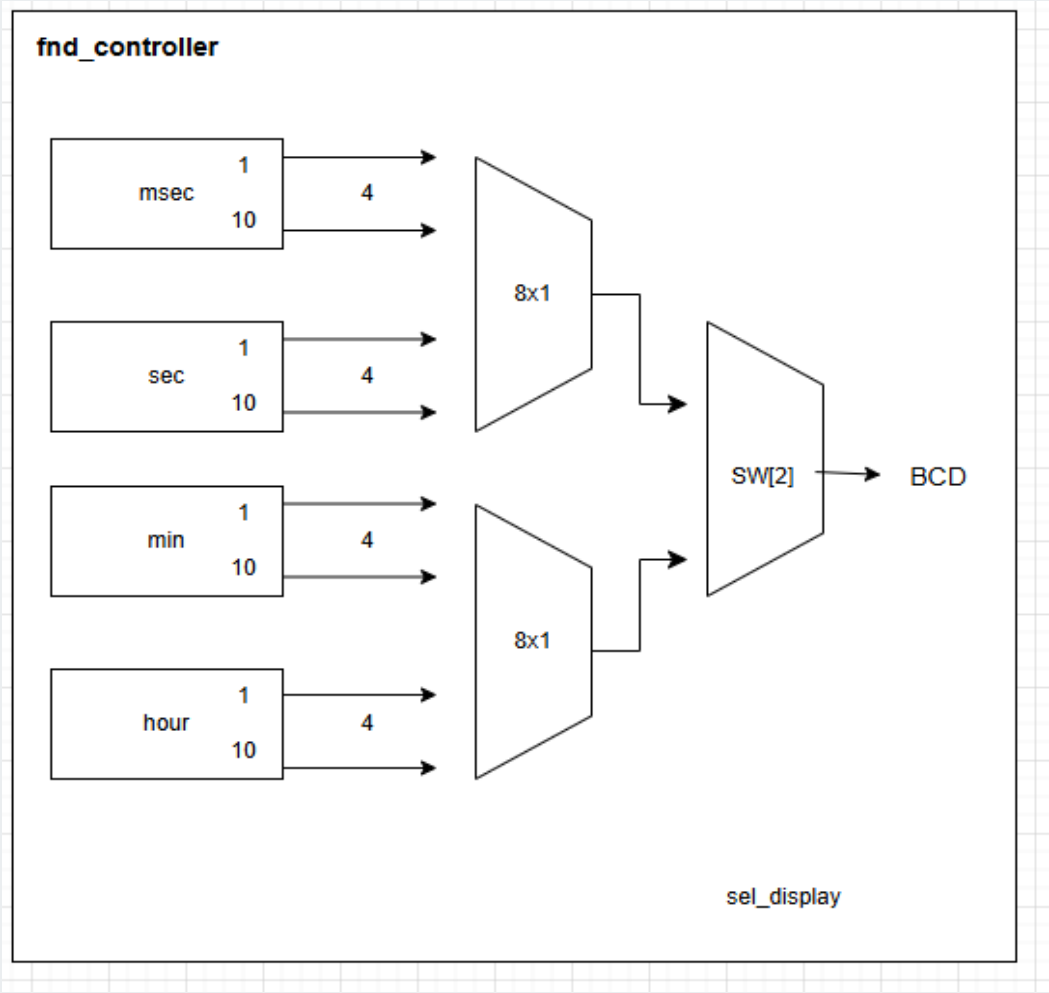
Watch path도 동일한 구조로 설계

시스템 구조

Watch/StopWatch path 에서 온 입력들을 받아 출력한다

제한된 4자리 FND에 표시하기 위해
MUX 활용 > SW[2]로 Hour_min / Sec_msec 전환

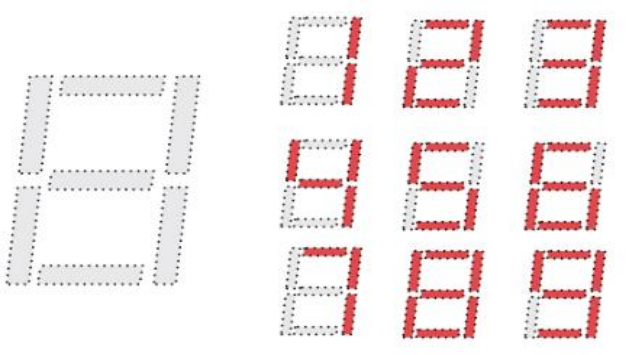
Sw[2] = = 0 (시 : 분) 선택
Sw[2] = = 1 (초 : 밀리초) 선택

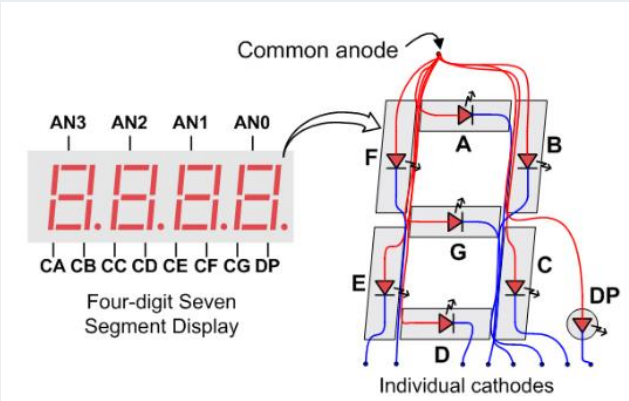


시스템 구조

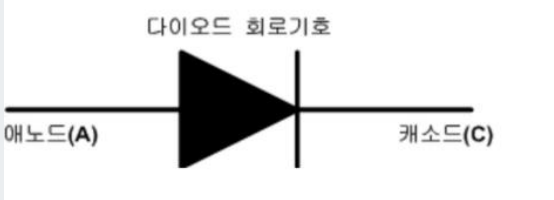
fnd_controller 출력 부 : segment-7

- 숫자 0 ~9를 보여주기 위해 막대 LED 7개로 만든 숫자 표시기

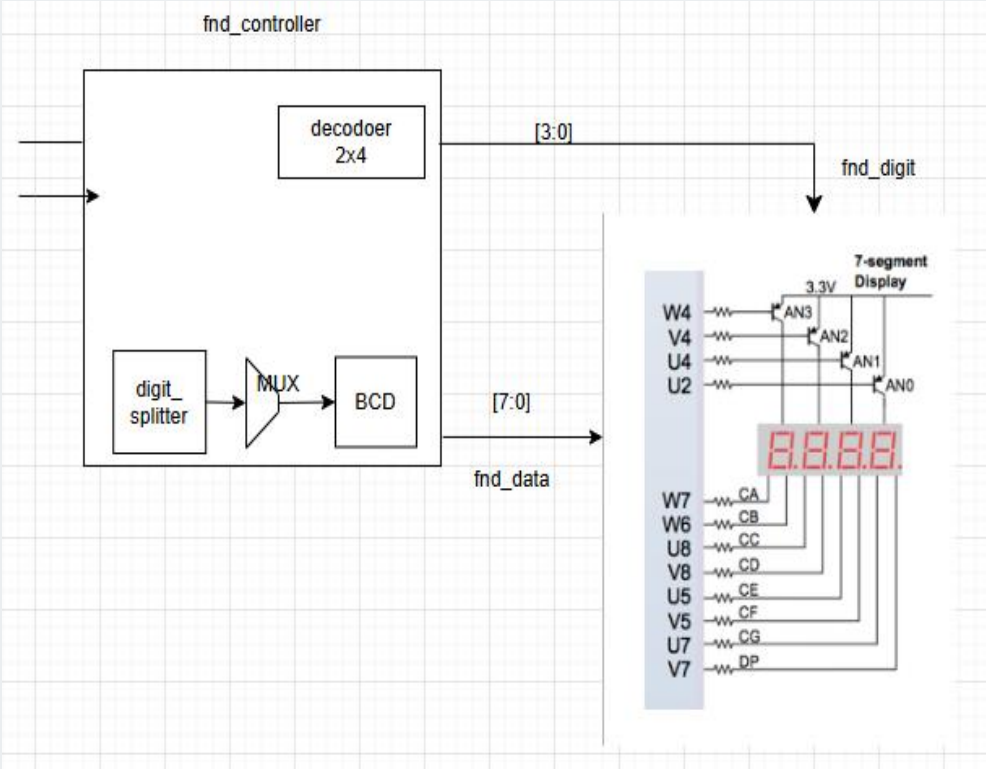




캐소드쪽을 사용하여 제어함
(다이오드는 애노드 -> 캐소드 방향으로
전류가 흐를 때만 켜진다)



시스템 구조

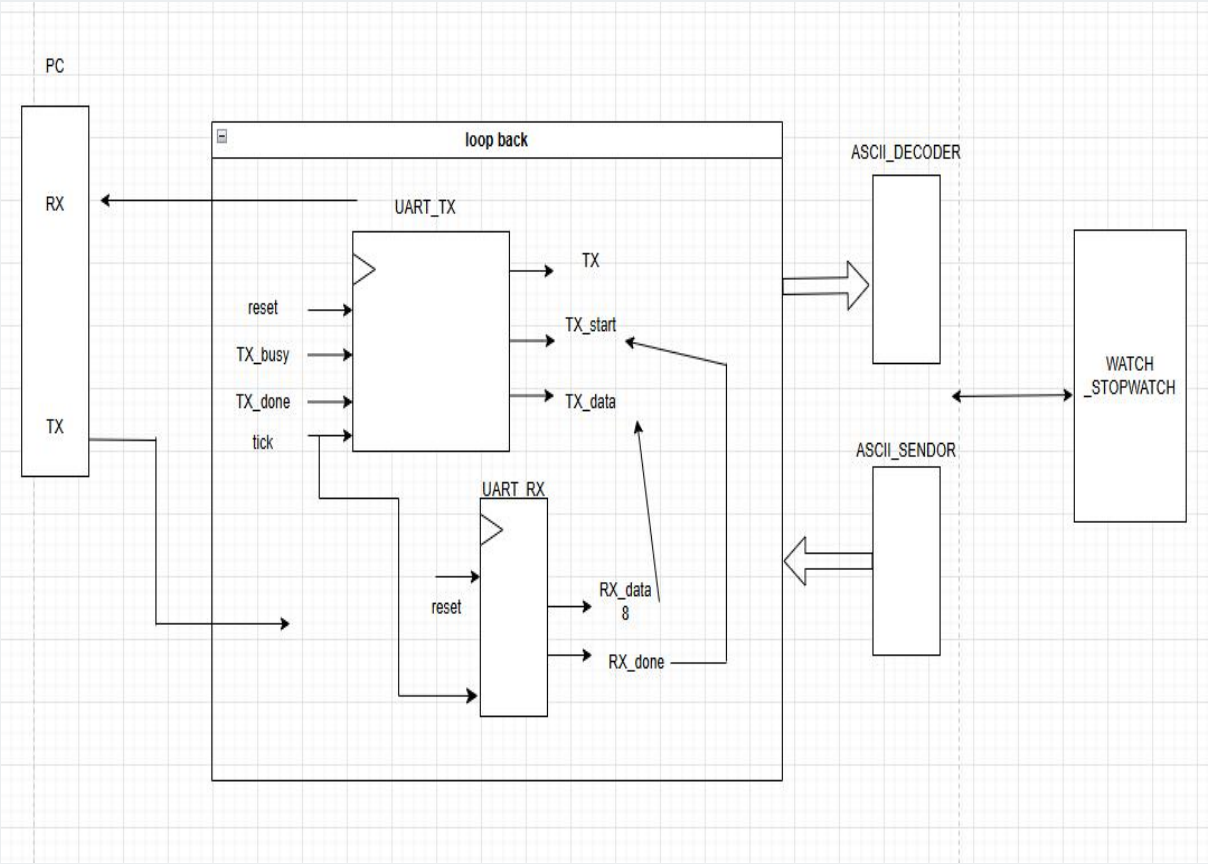


- 2x4 Decoder를 통해 2비트 카운터 값을 받아 4개의 자릿수 중 하나 선택
-> 4비트 fnd_digit

- 현재 선택된 자릿수에 표출할 숫자를 7-segment 패턴으로 변환
-> 8비트 fnd_data

시스템 구조

전체 통신 구조



ASCII

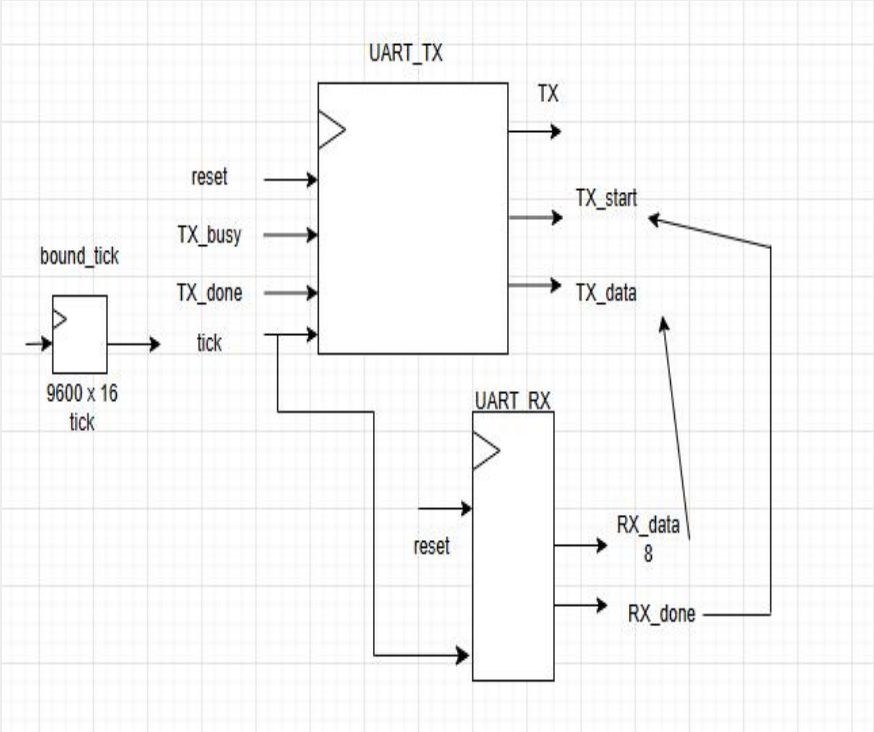
r : Btn R	0 : SW[0]
l : Btn L	1 : SW[1]
u : Btn up	2 : SW[2]
d : Btn down	s : FPGA 값 출력

ASCII_DECODER : PC(UART) -> FPGA로 다이얼 기능 제어
ASCII_SENDER : FPGA > PC(UART) 로 현재 FPGA(WATCH) 정보 확인

시스템 구조

RX data를 TX 모듈로 전달하여 PC로 되돌리는 루프 백 구조

9600bps x 16배수의 tick 신호를 TX 와 RX에 공통으로 입력



상세 설계

최상단 모듈

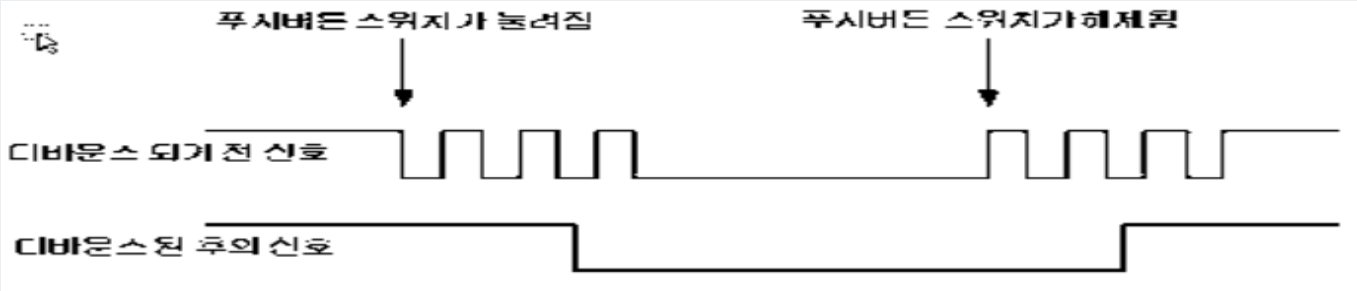
```
module system_top (
    input clk,
    input reset,
    input uart_rx,
    output uart_tx,
    input [2:0] sw,
    output [3:0] fnd_digit,
    output [7:0] fnd_data
);
  wire w_btn_r, w_btn_l, w_btn_u, w_btn_d;

  uart_top U_UART (
      .clk(clk),
      .reset(reset),
      .uart_rx(uart_rx),
      .uart_tx(uart_tx),
      .o_btn_r(w_btn_r),
      .o_btn_l(w_btn_l),
      .o_btn_u(w_btn_u),
      .o_btn_d(w_btn_d)
  );

  top_stopwatch_watch U_WATCH (
      .clk(clk),
      .reset(reset),
      .sw(sw),
      .btn_r(w_btn_r),
      .btn_l(w_btn_l),
      .btn_u(w_btn_u),
      .btn_d(w_btn_d),
      .fnd_digit(fnd_digit),
```

UART와 WATCH_STOPWATCH 연결할 수 있는 모듈 추가

상세 설계

Button Debounce

디바운싱 원리 : 기계적 스위치를 누를 때 접점이 붙었다 떨어지기를 반복하는 **채터링** 현상이 발생하는데 이를 제거하기 위해 일정 시간 동안 신호가 안정적으로 유지되는지 확인



상세 설계

Button Debounce

```
module bt_debounce (
    input  clk,
    input  reset,
    input  i_btn,
    output o_btn
);
    parameter CLK_DIV = 100_000;
    parameter F_COUNT = 100_000_000 / CLK_DIV;
    reg [$clog2(F_COUNT)-1:0] counter_reg;
    reg clk_100khz;

    always @(posedge clk, posedge reset) begin
        if (reset) begin
            counter_reg <= 0;
            clk_100khz  <= 1'b0;
        end else begin
            counter_reg <= counter_reg + 1;
            clk_100khz  <= 1'b0;
            if (counter_reg == F_COUNT - 1) begin
                counter_reg <= 0;
                clk_100khz  <= 1'b1;
            end else begin
                clk_100khz <= 1'b0;
            end
        end
    end
```

FPGA 의 속도는 매우 빠르기 때문에 미세한 노이즈 감지
>> 버튼을 수십 번 누른 것처럼 오작동

클럭을 Divide하여 속도를 늦춘다

100MHZ / 1000 >> 100kHz 주기 펄스 생성

0~999 까지 총 1000번을 1주기 신호로 내보내 Clk 계산

상세 설계

Button Debounce

```
reg [7:0] q_reg, q_next;
wire debounce;

always @(posedge clk_100khz, posedge reset) begin
    if (reset) begin
        q_reg <= 0;
    end else begin
        q_reg <= q_next;
    end
end

always @(*) begin
    q_next = {i_btn, q_reg[7:1]};
end

assign debounce = &q_reg;
reg edge_reg;

always @(posedge clk, posedge reset) begin
    if (reset) begin
        edge_reg <= 1'b0;
    end else begin
        edge_reg <= debounce;
    end
end
assign o_btn = debounce & (~edge_reg);
ndmodule
```

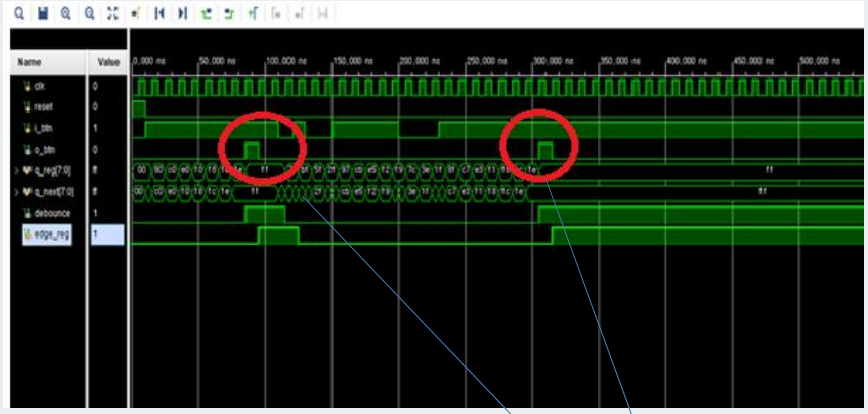
{ } 를 이용해 새로운 입력 버튼(i_btn) 계속 넣는 중
> 8칸 대기 열 만들고 기록

&q_reg
: 저장된 8개의 비트가 모두 1일 때만 Debounce 신호를 1로 만듦

버튼을 꾹 눌러도 한 번 만 신호를 보내줌

상세 설계

Button debounce 시뮬레이션

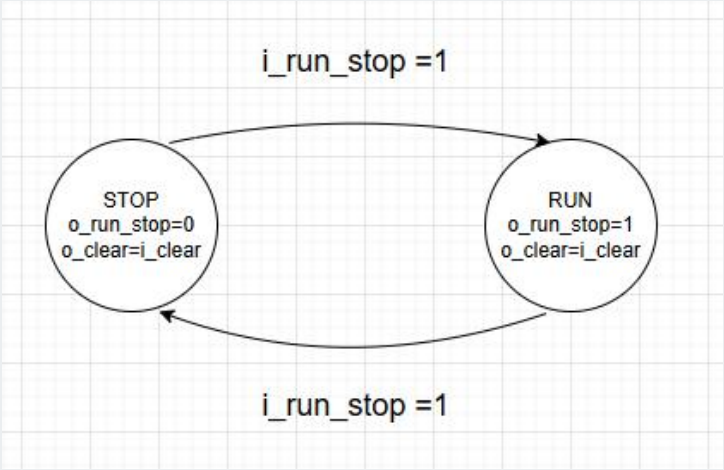


한 번이 아니라 연속된 비트가 1이어야 출력을 내보내는 것을 확인

상세 설계

Stopwatch_control_unit

```
    localparam STOP = 2'b00, RUN = 2'b01;
    reg current_state, next_state;
    assign o_mode = i_mode;

    always @(posedge clk, posedge reset) begin
        if (reset) begin
            current_state <= STOP;
        end else begin
            current_state <= next_state;
        end
    end

    always @(*) begin
        next_state = current_state;
        o_runstop = 1'b0;
        o_clear = i_clear;
        case (current_state)
            STOP: begin
                o_runstop = 1'b0;

                if (i_runstop) begin
                    next_state = RUN;
                end

            end
            RUN: begin
                o_runstop = 1'b1;

                if (i_runstop) begin
                    next_state = STOP;
                end
            end
        endcase
    end
endmodule
```



2개의 상태만을 이용한 FSM 구조

o_clear = i_clear;
명령어를 case 문 밖에 배치하여
초기화 기능과 상관없이
작동하도록 설계

상세 설계

자릿수 분리

```
endmodule

module digit_splitter #(
    parameter BIT_WIDTH = 7
) (
    input [BIT_WIDTH-1:0] in_data,
    output [3:0] digit_1,
    output [3:0] digit_10
);

    assign digit_1  = in_data % 10;
    assign digit_10 = (in_data / 10) % 10;

endmodule
```

Hour(5비트), min/sec(6비트), msec(7비트)
의 크기의 데이터를 받기 위해 수학적 연산
기호를 활용해 두 개의 십진수 자릿수로 분리

fnd_controller

점 깜빡임 제어

```
module dot_on_comp (
    input [6:0] msec,
    output dot_onoff
);

    assign dot_onoff = (msec < 50);

endmodule
```

밀리초는 0~99까지 도는 것을 활용해
규칙을 설정해서 0.5초에 한 번씩 규칙적으로
깜빡이는 효과를 만듦

상세 설계

fnd_controller

자릿수 카운터

```
module counter_8 (
    input clk,
    input reset,
    output [2:0] digit_sel
);

    reg [2:0] counter_r;
    assign digit_sel = counter_r;

    always @(posedge clk, posedge reset) begin
        if (reset) begin
            counter_r <= 0;
        end else begin
            counter_r <= counter_r + 1;
        end
    end
endmodule
```

1kHz 박자에 맞춰 숫자를 센다
현재 몇 번째 FND 자리를 키는지 결정

자릿수 선택 신호 변환

```
//to select to fnd digit display
module decoder_2x4 (
    input [1:0] digit_sel,
    output reg [3:0] fnd_digit

);

    always @(digit_sel) begin
        case (digit_sel)
            2'b00: fnd_digit = 4'b1110;
            2'b01: fnd_digit = 4'b1101;
            2'b10: fnd_digit = 4'b1011;
            2'b11: fnd_digit = 4'b0111;
        endcase

    end

endmodule
```

카운터가 보낸 숫자를 받아서 실제 FND의
Anode(전원 핀)을 제어하는 신호로 변경

상세 설계

```
module mux_8x1 (
    input       [2:0] sel,
    input       [3:0] digit_1,
    input       [3:0] digit_10,
    input       [3:0] digit_100,
    input       [3:0] digit_1000,
    input       [3:0] digit_dot_1,
    input       [3:0] digit_dot_10,
    input       [3:0] digit_dot_100,
    input       [3:0] digit_dot_1000,
    output reg  [3:0] mux_out
);

    always @(*) begin
        case (sel)
            3'b000: mux_out = digit_1;
            3'b001: mux_out = digit_10;
            3'b010: mux_out = digit_100;
            3'b011: mux_out = digit_1000;
            3'b100: mux_out = digit_dot_1;
            3'b101: mux_out = digit_dot_10;
            3'b110: mux_out = digit_dot_100;
            3'b111: mux_out = digit_dot_1000;
        endcase
    end

endmodule
```

켜진 자리에 해당하는 숫자 데이터를 고름

fnd_controller

Mux를 이용한 데이터 선택

```
module mux2x1 (
    input sel,
    input [3:0] i_sel0,
    input [3:0] i_sel1,
    output [3:0] o_mux
);

    assign o_mux = (sel) ? i_sel1 : i_sel0;

endmodule
```

(시 : 분) 출력할지 **(초 : 밀리초)** 일지 선택

상세 설계

fnd_controller

```
module bcd (
    input       [3:0] bcd,
    output reg [7:0] fnd_data
);

    always @(bcd) begin
        case (bcd)
            4'd0: fnd_data = 8'hC0;
            4'd1: fnd_data = 8'hf9;
            4'd2: fnd_data = 8'ha4;
            4'd3: fnd_data = 8'hB0;
            4'd4: fnd_data = 8'h99;
            4'd5: fnd_data = 8'h92;
            4'd6: fnd_data = 8'h82;
            4'd7: fnd_data = 8'hf8;
            4'd8: fnd_data = 8'h80;
            4'd9: fnd_data = 8'h90;
            4'd10: fnd_data = 8'hff;
            4'd11: fnd_data = 8'hff;
            4'd12: fnd_data = 8'hff;
            4'd13: fnd_data = 8'hff;
            4'd14: fnd_data = 8'h7f;
            4'd15: fnd_data = 8'hff;
            default: fnd_data = 8'hFF;
        endcase

    end

endmodule
```

최종 출력 변환

선택된 숫자(4비트)를 받아서 LED 전구를 켜고 끄는 패턴으로 바꿔준다.

상세 설계

```
wire w_runstop, w_clear, w_mode;
wire o_btn_runstop, o_btn_clear;
wire [23:0] w_stopwatch_time;
wire [23:0] w_watch_time;
reg  [23:0] w_fnd_in_data;
wire db_btn_r, db_btn_l, db_btn_u, db_btn_d;

wire sel_mode = sw[1];

assign o_btn_runstop = (sel_mode == 1'b1) ? db_btn_r : 1'b0;
assign o_btn_clear   = (sel_mode == 1'b1) ? db_btn_l : 1'b0;

wire w_btn_left = (sel_mode == 1'b0) ? db_btn_l : 1'b0;
wire w_btn_right = (sel_mode == 1'b0) ? db_btn_r : 1'b0;
wire w_btn_up = (sel_mode == 1'b0) ? db_btn_u : 1'b0;
wire w_btn_down = (sel_mode == 1'b0) ? db_btn_d : 1'b0;
```

4개의 버튼을 이용하여 모드 변경
Sel_mode 신호를 통해 스톱워치 모드는 기능
시계모드에서는 방향키와 크기 조절

Top Module

```
wire [23:0] w_stopwatch_time;
wire [23:0] w_watch_time;
reg  [23:0] w_fnd_in_data;
```

```
always @(*) begin
  if (sel_mode == 1'b1) w_fnd_in_data = w_stopwatch_time;
  else w_fnd_in_data = w_watch_time;
end
```

스톱워치/시계 모드 변경

상세 설계

```
reg [1:0] cursor;
reg prev_l, prev_r, prev_u, prev_d;
wire edge_l = btn_l & ~prev_l;
wire edge_r = btn_r & ~prev_r;
wire edge_u = btn_u & ~prev_u;
wire edge_d = btn_d & ~prev_d;

always @(posedge clk or posedge reset) begin
  if (reset) begin
    cursor <= 0;
    prev_l <= 0;
    prev_r <= 0;
    prev_u <= 0;
    prev_d <= 0;
  end else begin
    prev_l <= btn_l;
    prev_r <= btn_r;
    prev_u <= btn_u;
    prev_d <= btn_d;

    if (edge_r) begin
      if (cursor == 0) cursor <= 3;
      else cursor <= cursor - 1;
    end else if (edge_l) begin
      if (cursor == 0) cursor <= 1;
      else if (cursor == 3) cursor <= 0;
      else cursor <= cursor + 1;
    end
  end
end
```

Watch_datapath

Cursor 변수 도입하여 시계 수정 설계

우측 버튼을 누를 때 마다
시 > 분 > 초 순서로 수정 모드 진입 가능하며
위/아래 버튼을 이용해 제어 가능

```
wire min_tick = (cursor == 2) ? manual_tick : (w_sec_carry & (cursor != 1));
wire min_mode = (cursor == 2) ? manual_mode : 1'b0;
```

수정 하고 있을 때는 올림 수 안받게 설계

상세 설계

```
module tick_counter #(
    parameter BIT_WIDTH = 7,
    TIMES = 100,
    RESET_VALUE = 0
) (
    input clk,
    input reset,
    input i_tick,
    input mode,
    input clear,
    input run_stop,
    output [BIT_WIDTH-1:0] o_count,
    output reg o_tick
);
  reg [BIT_WIDTH-1:0] counter_reg, counter_next;
  assign o_count = counter_reg;

  always @(posedge clk, posedge reset) begin
    if (reset | clear) begin
      counter_reg <= RESET_VALUE;
    end else begin
      counter_reg <= counter_next;
    end
  end
```

datapath

```
  always @(*) begin
    counter_next = counter_reg;
    o_tick = 1'b0;
    if (i_tick & run_stop) begin
      if (mode == 1'b1) begin

        if (counter_reg == 0) begin
          counter_next = TIMES - 1;
          o_tick = 1'b1;
        end else begin
          counter_next = counter_reg - 1;
          o_tick = 1'b0;
        end
      end else begin
        if (counter_reg == (TIMES - 1)) begin
          counter_next = 0;
          o_tick = 1'b1;
        end else begin
          counter_next = counter_reg + 1;
          o_tick = 1'b0;
        end
      end
    end
  end
endmodule
```

Parameter 기능을 활용하여 모듈 하나로 100진법(밀리초), 60진법(초,분), 24진법(시) 구현

상세 설계

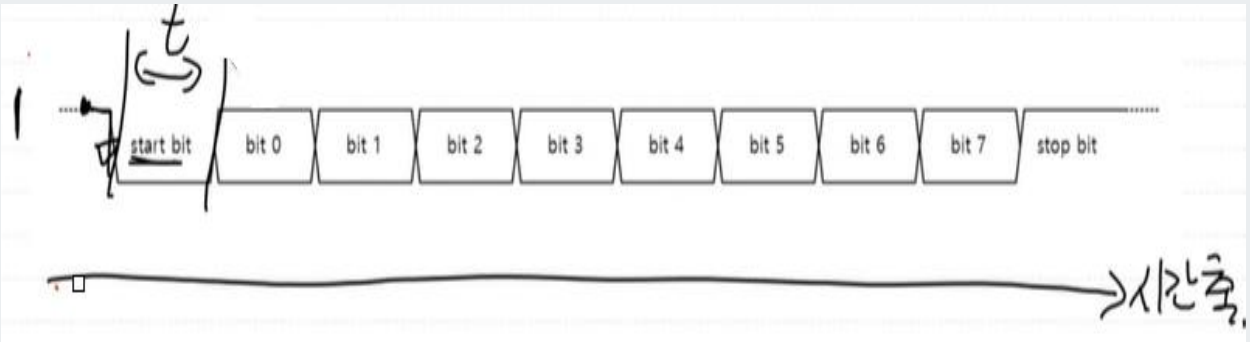
datapath

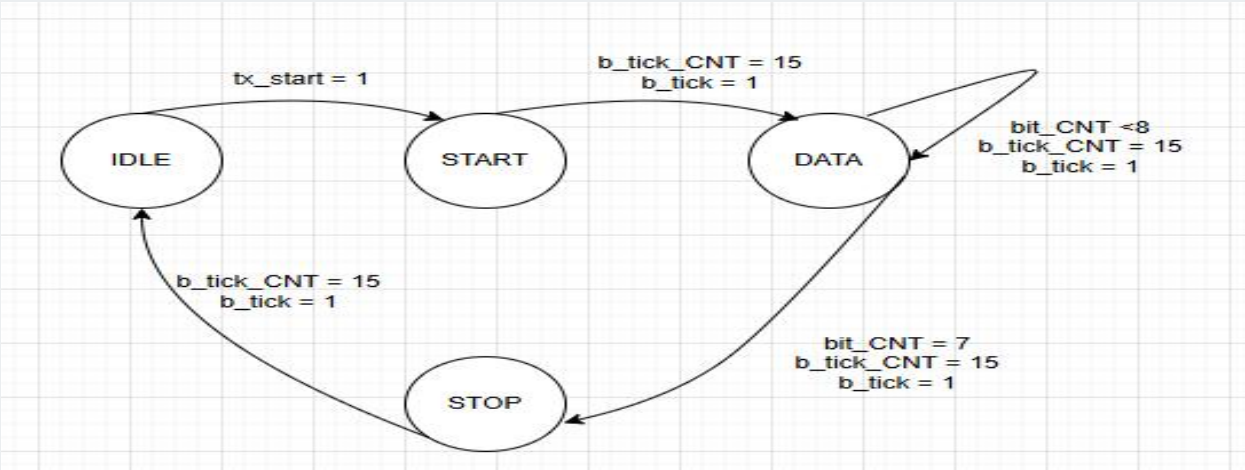
```
stopwatch_datapath U_STOPWATCH_DATAPATH (
    .clk(clk),
    .reset(reset),
    .mode(w_mode),
    .clear(w_clear),
    .run_stop(w_runstop),
    .msec(w_stopwatch_time[6:0]),
    .sec(w_stopwatch_time[12:7]),
    .min(w_stopwatch_time[18:13]),
    .hour(w_stopwatch_time[23:19])
);

watch_datapath U_WATCH_DATAPATH (
    .clk  (clk),
    .reset(reset),
    .btn_l(w_btn_left),
    .btn_r(w_btn_right),
    .btn_u(w_btn_up),
    .btn_d(w_btn_down),
    .msec (w_watch_time[6:0]),
    .sec  (w_watch_time[12:7]),
    .min  (w_watch_time[18:13]),
    .hour (w_watch_time[23:19])
);
```

Top 모듈에서 스톱워치와 시계 모듈을 인스턴스화 하여 나란히 선언
두 시스템이 독립적이고 병렬적으로 동작하며 SW로 화면에 제어하도록 최종적으로 설계 완료

상세 설계

UART(TX)

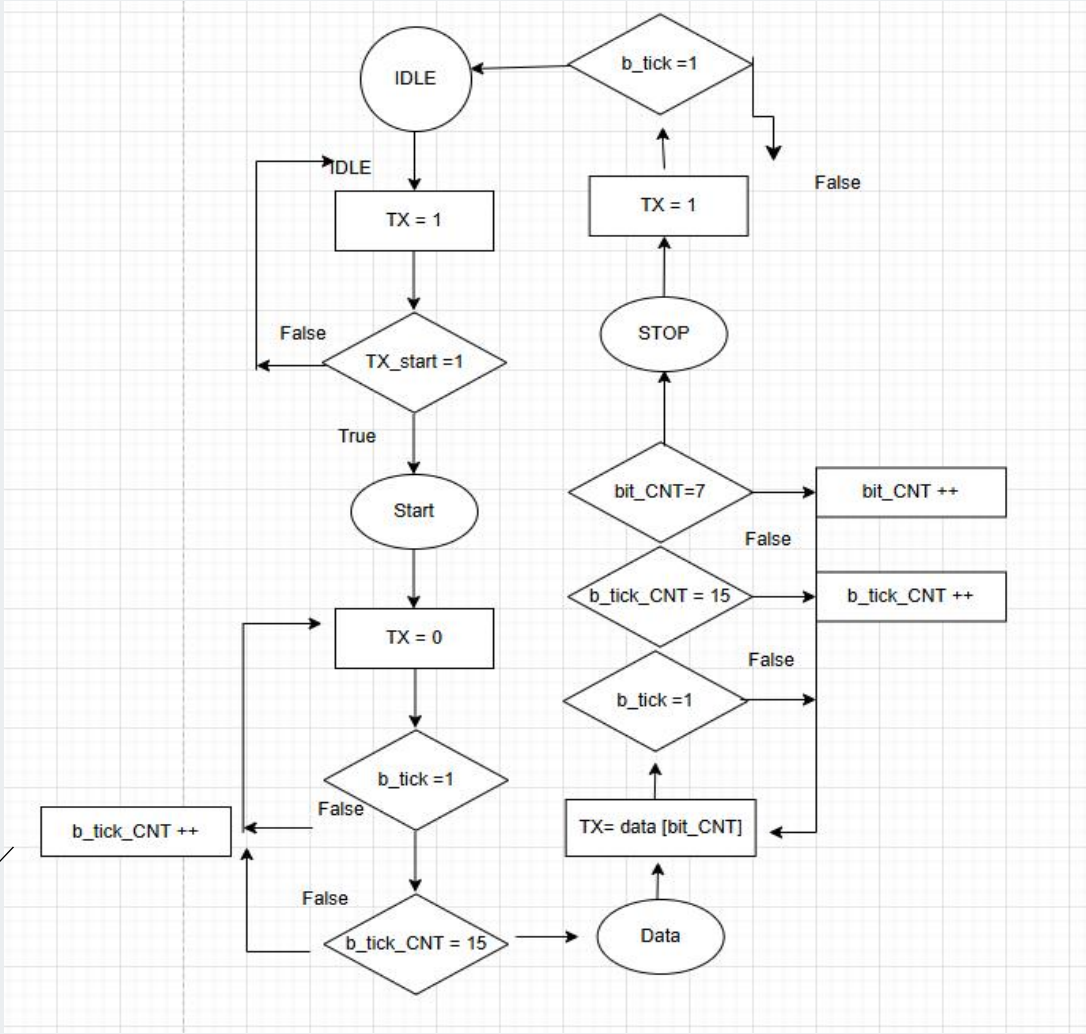

t
start bit
bit 0
bit 1
bit 2
bit 3
bit 4
bit 5
bit 6
bit 7
stop bit
시간축


tx_start = 1
b_tick_CNT = 15
b_tick = 1
IDLE
START
DATA
bit_CNT <8
b_tick_CNT = 15
b_tick = 1
b_tick_CNT = 15
b_tick = 1
bit_CNT = 7
b_tick_CNT = 15
b_tick = 1
STOP

상세 설계

UART(TX)

알고리즘 구조도



해당 값을 충족 못하면 계속 증가

상세 설계

UART(TX)

```
always @(*) begin
    n_state = c_state;
    tx_next = tx_reg;
    bit_cnt_next = bit_cnt_reg;
    b_tick_cnt_next = b_tick_cnt_reg;
    busy_next = busy_reg;
    done_next = done_reg;
    data_in_buf_next = data_in_buf_reg;


    case (c_state)
        IDLE: begin
            tx_next = 1'b1;
            bit_cnt_next = 1'b0;
            b_tick_cnt_next = 4'h0;
            busy_next = 1'b0;
            done_next = 1'b0;
            if (tx_start) begin
                n_state = START;
                busy_next = 1'b1;
                data_in_buf_next = tx_data;

            end
        end

        START: begin

            tx_next = 1'b0;
            if (b_tick) begin
                if (b_tick_cnt_reg == 15) begin
                    n_state = DATA;
                    b_tick_cnt_next = 4'h0;
                end else begin
                    b_tick_cnt_next = b_tick_cnt_reg + 1;
                end
            end
        end

        DATA: begin
            tx_next = data_in_buf_reg[0];
            if (b_tick) begin
                if (b_tick_cnt_reg == 15) begin
                    if (bit_cnt_reg == 7) begin
                        b_tick_cnt_next = 4'h0;
                        n_state = STOP;
                    end else begin
                        b_tick_cnt_next = 4'h0;
                        bit_cnt_next = bit_cnt_reg + 1;
                        data_in_buf_next = {1'b0, data_in_buf_reg[7:1]};
                        n_state = DATA;
                    end
                end else begin
                    b_tick_cnt_next = b_tick_cnt_reg + 1;
                end

            end
        end


        STOP: begin
            tx_next = 1'b1;
            if (b_tick) begin
                if (b_tick_cnt_reg == 15) begin
                    done_next = 1'b1;
                    n_state   = IDLE;

                end else begin
                    b_tick_cnt_next = b_tick_cnt_reg + 1;
                end
            end
        end
    endcase
end
```

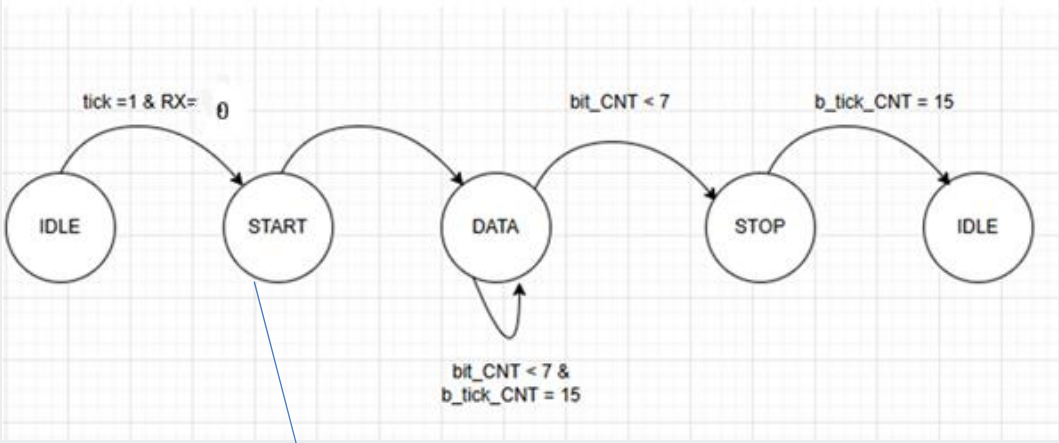
Parallel 구조 이므로
Shift register 활용

Case문을 활용하여 설계한 구조도 대로 코드 구현

상세 설계

UART(RX)

Rx를 감시하며 대기
0으로 떨어지는 구간에서 START



비트의 정중앙 시점에서 rx값 다시 확인하여 0이면 DATA

상세 설계

UART(RX)

```
always @(*) begin
  n_state = c_state;
  b_tick_cnt_next = b_tick_cnt_reg;
  bit_cnt_next = bit_cnt_reg;
  done_next = done_reg;
  buf_next = buf_reg;
  case (c_state)
    IDLE: begin
      b_tick_cnt_next = 5'd0;
      bit_cnt_next = 3'd0;
      done_next = 1'b0;
      if (b_tick & rx == 0) begin
        buf_next = 8'd0;
        n_state  = START;
      end
    end
    START: begin
      if (b_tick)
        if (b_tick_cnt_reg == 7) begin
          b_tick_cnt_next = 0;
          n_state = DATA;
        end else begin
          b_tick_cnt_next = b_tick_cnt_reg + 1;
        end
    end
```

```
START: begin
  if (b_tick)
    if (b_tick_cnt_reg == 7) begin
      b_tick_cnt_next = 0;
      n_state = DATA;
    end else begin
      b_tick_cnt_next = b_tick_cnt_reg + 1;
    end
end
DATA: begin
  if (b_tick) begin
    if (b_tick_cnt_reg == 15) begin
      b_tick_cnt_next = 0;
      buf_next = {rx, buf_reg[7:1]};
      if (bit_cnt_reg == 7) begin
        n_state = STOP;
      end else begin
        bit_cnt_next = bit_cnt_reg + 1;
      end
    end else begin
      b_tick_cnt_next = b_tick_cnt_reg + 1;
    end
  end
```

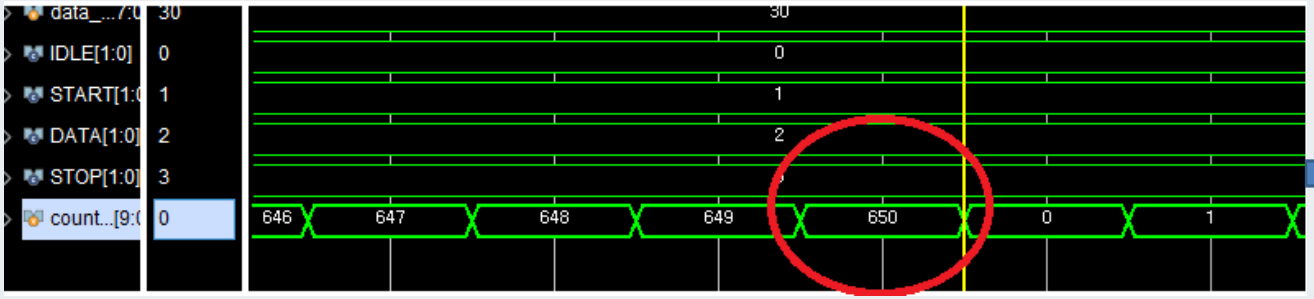
```
    STOP: begin
      if (b_tick)
        if (b_tick_cnt_reg == 15) begin
          n_state   = IDLE;
          done_next = 1'b1;
        end else begin
          b_tick_cnt_next = b_tick_cnt_reg + 1;
        end
    end
  endcase
end
```

LSB 방식
새로 들어온 비트를 가장 높은 자리
기존 비트들을 오른쪽으로 미는 방식

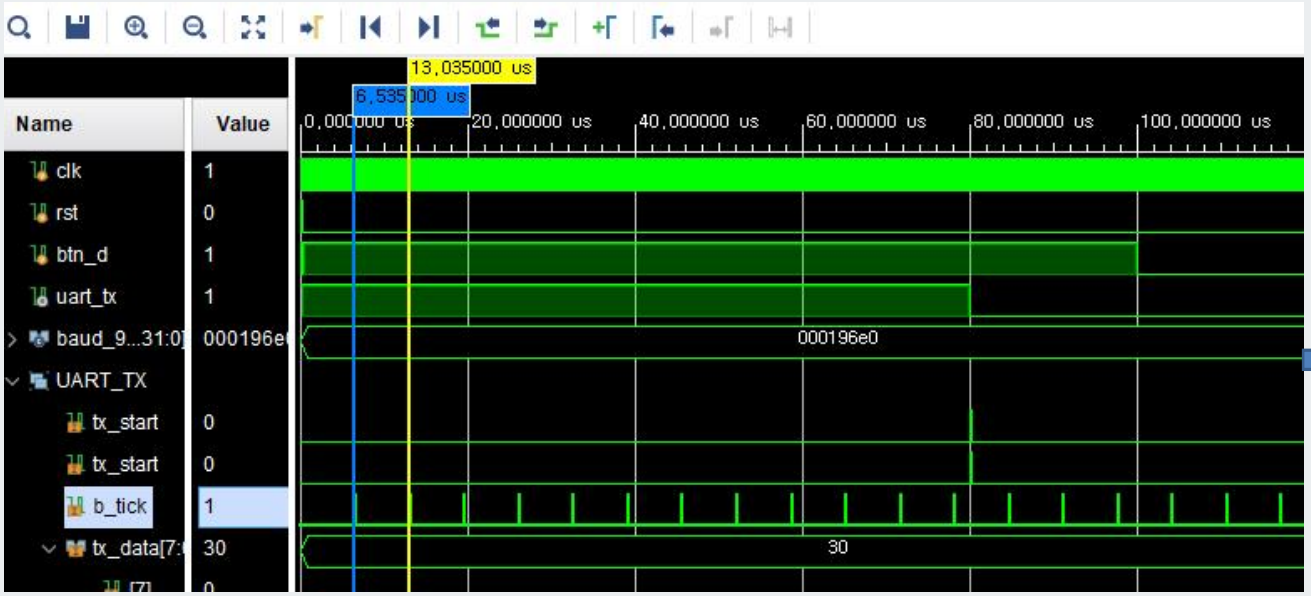
-> 모든 비트를 받아 rx_done을 1클럭 동안 high

상세 설계

UART 구조 시뮬레이션



16배속 650까지 도달



Tick 주기 약 6.51us

상세 설계

Decoder

```verilog
module ascii_decoder (
    input [7:0] rx_data,
    input rx_done,
    output reg btn_r,
    output reg btn_l,
    output reg btn_u,
    output reg btn_d
);

  always @(*) begin
    btn_r = 0;
    btn_l = 0;
    btn_u = 0;
    btn_d = 0;

    if (rx_done) begin
      if (rx_data == 8'h72) btn_r = 1;
      else if (rx_data == 8'h6C) btn_l = 1;
      else if (rx_data == 8'h75) btn_u = 1;
      else if (rx_data == 8'h64) btn_d = 1;
    end
  end

endmodule
```

누르는 키	16진수 (Hex)	Verilog 코드
r (Right)	**72**	8'h72
l (Left)	**6C**	8'h6C
u (Up)	**75**	8'h75
d (Down)	**64**	8'h64

PC에서 보내는 데이터를 변환하여 전달

상세 설계

SENDER

```
always @(posedge clk or posedge reset) begin
  if (reset) begin
    state <= IDLE;
    num   <= 0;
  end else begin
    state <= state_n;
    num   <= num_n;
  end
end

always @(*) begin
  state_n  = state;
  num_n    = num;
  tx_start = 1'b0;
  tx_data  = 8'h00;
```

```
case (state)
  IDLE: begin
    if (start) begin
      num_n   = 0;
      state_n = SEND;
    end
  end

  SEND: begin
    tx_data  = msg[num];
    tx_start = 1'b1;
    state_n  = WAIT;
  end

  WAIT: begin
    if (tx_done) begin
      if (num == 9) state_n = IDLE;
      else begin
        num_n   = num + 1;
        state_n = SEND;
      end
    end
  end
endcase
```

IDLE-SEND-WAIT 상태 전이를 통해 start와 done 신호를 주고받는 **FSM** 구조를 구현하여 데이터를 전송

상세 설계

SENDER

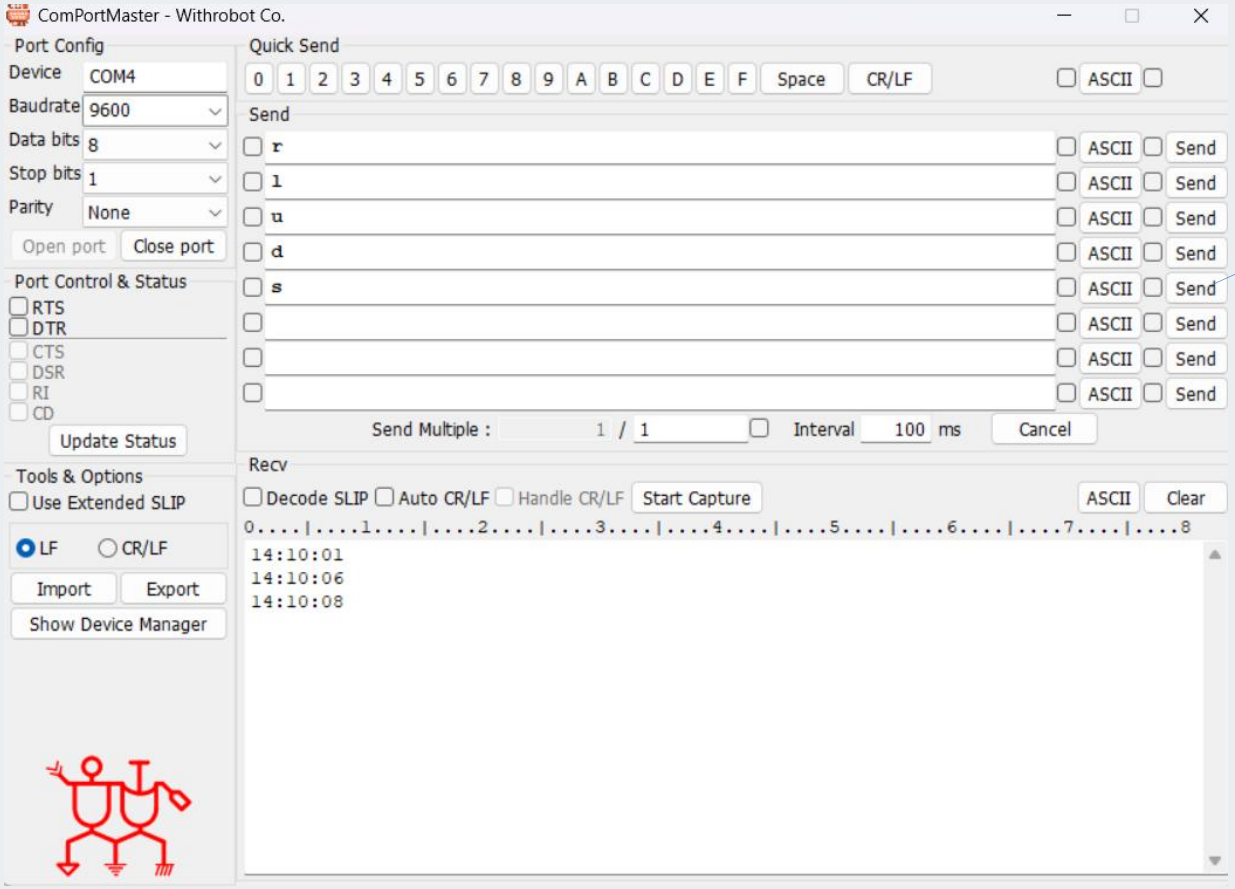
```
wire [4:0] cur_hour = w_fnd_in_data[23:19];
wire [5:0] cur_min = w_fnd_in_data[18:13];
wire [5:0] cur_sec = w_fnd_in_data[12:7];
wire [6:0] cur_msec = w_fnd_in_data[6:0];


assign o_hour_10 = cur_hour / 10;
assign o_hour_1  = cur_hour % 10;
assign o_min_10  = cur_min / 10;
assign o_min_1   = cur_min % 10;
assign o_sec_10  = cur_sec / 10;
assign o_sec_1   = cur_sec % 10;
assign o_msec_10 = cur_msec / 10;
assign o_msec_1  = cur_msec % 10;
```

ASCII 문자를 인식하는 PC터미널과 통신하기 위해
5~7 비트에 해당하는 시간 값을
10의 자리와 1의 자리로 분리하는 **BCD 변환**

단순 비트 슬라이싱이 아닌 나눗셈과 나머지 연산을
통해 실시간 데이터를 10진수 자릿수 별로
독립적인 데이터 스트림으로 분리

기능 검증

SENDOR



“s” 값을 보내면
FPGA에 있는
시간 정보 출력되는
것을 확인

영상 시연

Watch(fpga) : https://youtube.com/shorts/DJeXl8HS0sM

Stopwatch(fpga) : https://youtube.com/shorts/R1xkWumHBts

Watch(UART) : https://youtu.be/m0dN-eH3NqQ

Stopwatch(UART) : https://youtu.be/KijPGcPV92Q

Trouble Shooting

UART Loopback 테스트(PC → FPGA → PC)는 정상적으로 동작함 (데이터 수신 확인)
그러나 **스톱워치 모듈이 해당 신호에 전혀 반응하지 않음**

원인 -> btn debounce : 직접 입력할 때는 필수적이었지만 UART에서는 통신 방해 일으킴

```
assign o_btn_runstop = (sel_mode == 1'b1) ? (db_btn_r | btn_r) : 1'b0;
assign o_btn_clear   = (sel_mode == 1'b1) ? (db_btn_l | btn_l) : 1'b0;

wire w_btn_left = (sel_mode == 1'b0) ? (db_btn_l | btn_l) : 1'b0;
wire w_btn_right = (sel_mode == 1'b0) ? (db_btn_r | btn_r) : 1'b0;
wire w_btn_up = (sel_mode == 1'b0) ? (db_btn_u | btn_u) : 1'b0;
wire w_btn_down = (sel_mode == 1'b0) ? (db_btn_d | btn_d) : 1'b0;
```

Or 문을 활용하여
Debounce 우회

결론

1. 프로젝트 성과

- 단일 FPGA 보드에서 시계와 스톱워치 두 가지 독립 된 시스템 통합 제어
- FSM 기반의 양방향 UART 통신 모듈을 설계
- Debouncing 과 Edge detection을 통해 노이즈가 없는 입력 시스템 구축
- 16배수 오버 샘플링 기법과 중앙 샘플링 알고리즘을 적용 노이즈 제거

2. 향후 발전 과제

- PC 와 시리얼 통신을 통해 현재 시간을 자동으로 동기화하는 기능 고민